

Coding Assignment 6: P1 – Delete at Position, P2 – Merge Sorted Lists

Auqib Hamid Lone

Department of Computer Science & Engineering, GCET Ganderbal

September 26, 2026

Due: two weeks from issue. There is no automatic late penalty — if you need more time, ask before the deadline and an extension will be granted (see the course policy in the syllabus).

This assignment autogrades **two** institute practicals on Week 6’s singly linked list (**head** is the list handle, **p->next** is the successor link, **NULL** terminates the chain): **P1** (delete the node at a 0-based position) and **P2** (merge two sorted lists into one). The contract for both is given to you in **sll.h** — *do not modify it*. Helpers (**newNode**, **buildList**, **freeList**, **listToArray**, **listLength**) are implemented in **sll_common.c** — *do not modify it either*. Each part is a separate program with its own starter file and hidden test suite.

6.1 Why this problem

The lecture built deletion by head and deletion by value, and traced reversal — but never deletion *by position* (drop the k -th entry of a log without knowing its value) and never *merging* two ordered chains (combine two sorted runs without allocating anything new). Both are the same pointer surgery the lecture drilled — save, unlink, free (P1); adopt-a-successor rewiring (P2) — applied to the cases the lecture’s own traces did not cover.

6.2 Part 1 (P1): Delete at Position

Implement in **p1.c** (start from **p1_starter.c**):

```
int deleteAtPosition(struct node **head, int pos);
```

pos is a 0-based index: **pos == 0** removes the head (the lecture’s save/advance/free triple), **pos > 0** walks to the predecessor exactly as the lecture’s **deleteValue** walk does, then unlinks and frees. Return 0 on success. Return **-1** and change **nothing** (no unlink, no free) when the list is empty, **pos** is negative, or **pos** is at or past the end of the list.

6.2.1 Constraints

- The parameter must stay **struct node **head**: removing position 0 reassigns the caller’s **head**, exactly the lecture’s double-pointer discipline.
- Unlink *before* freeing — the predecessor must point around the victim first (lecture’s use-after-free rule).

- Time $O(n)$ worst case, $n = \text{list length}$ (the walk dominates); auxiliary space $O(1)$.
- Your code must compile with `gcc -Wall -Wextra` without errors or warnings, with no global/static mutable state.

6.2.2 Worked examples (these are the visible tests)

All on the list `head -> 7 -> 13 -> 19 -> 27 -> NULL`:

- `deleteAtPosition(&h, 1)` returns 0; the list becomes `7 -> 19 -> 27 -> NULL`.
- `deleteAtPosition(&h, 0)` returns 0; the list becomes `13 -> 19 -> 27 -> NULL`.
- `deleteAtPosition(&h, 3)` returns 0; the list becomes `7 -> 13 -> 19 -> NULL`.
- `deleteAtPosition(&h, 4)` returns `-1`; the list is unchanged.
- A singleton list `42 -> NULL` with `pos = 0` returns 0 and leaves `h == NULL`.

6.3 Part 2 (P2): Merge Two Sorted Lists

Implement in `p2.c` (start from `p2_starter.c`):

```
struct node *mergeSorted(struct node *a, struct node *b);
```

Both inputs are ascending (either may be `NULL` for empty). **Reuse the existing nodes**: rewire `next` links only — allocate nothing, free nothing, so every input node appears exactly once in the result. Return the head of the merged list (`NULL` only when both inputs are empty). On ties (`a->data == b->data`), take the node from `a` first. After the call, the old `a/b` heads are owned by the merged list — do not use them.

6.3.1 Constraints

- Time $O(n + m)$ worst case ($n, m = \text{input lengths}$, each node visited once); auxiliary space $O(1)$ besides a few pointers.
- Your code must compile with `gcc -Wall -Wextra` without errors or warnings, with no global/static mutable state.

6.3.2 Worked examples (these are the visible tests)

- `mergeSorted([2, 8, 21], [5, 15, 33])` returns `2 -> 5 -> 8 -> 15 -> 21 -> 33 -> NULL`.
- `mergeSorted([3, 11, 29], [])` returns `3 -> 11 -> 29 -> NULL`.
- `mergeSorted([], [])` returns `NULL`.
- `mergeSorted([4, 9, 18], [9, 18, 26])` returns `4 -> 9 -> 9 -> 18 -> 18 -> 26 -> NULL`.

6.4 What to submit for each part

1. **P1:** `p1.c` (your completed implementation) and the unmodified `sll.h`, `sll_common.c`.
2. **P2:** `p2.c` (your completed implementation) and the unmodified `sll.h`, `sll_common.c`.

Sanity-check locally before submitting:

```
gcc -Wall -Wextra -o p1_test p1_test_visible.c p1.c sll_common.c && ./p1_test
gcc -Wall -Wextra -o p2_test p2_test_visible.c p2.c sll_common.c && ./p2_test
```

All worked examples above should pass. The autograder then compiles each part against its own *hidden* test suite. Each hidden suite covers, at minimum:

- the worked examples above, re-checked on fresh data;
- boundary sizes (empty/singleton/negative-position for P1; empty/singleton/tie for P2);
- structure-specific adversarial cases — for P1: head/tail/out-of-range/huge-position, draining a list to empty, all-duplicate lists, a 100-element list; for P2: one side entirely smaller, unequal lengths, negative values, duplicates within and across lists, a 500-node merge, sortedness plus node conservation on every merge;
- and a batch of randomly generated inputs checked against an independent oracle.